

Week 5

2. Concurrency in Java

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2800 – Software Development [2026W]

Wednesday, February 4th, 2026 | 14:30 – 15:50 | ER 1118



University of Windsor

Agenda



Concurrency in Java:

- **Processes vs. Threads**

- **Multithreading**

- **Lifecycle of a Thread**

- **Thread Priorities**

Part I

Processes and Threads



What is a Process?



Creating a new Process: `fork()`
Terminating a process: `exit()`
`wait()` and `waitpid()`

Context

Context

- single computer (single CPU)
- under UNIX operating system

It is possible to have several simultaneously executing programs

→ It appears that the executions are done in Parallel!

Multiprogramming:

The CPU executes some instructions of one program **then switches** to another program giving the **illusion** that any program is continuously executing, this scheme is called ***time-slicing***.

Creating a new Process: `fork()`
Terminating a process: `exit()`
`wait()` and `waitpid()`

Program vs. process

A *program* is an executable file residing on a disk.

A *process* is an executing (running) program, usually with a limited life-time.

Note that sometimes a process is called *task*.

Important notion

An executing program → a *process*.

Processes vs Threads



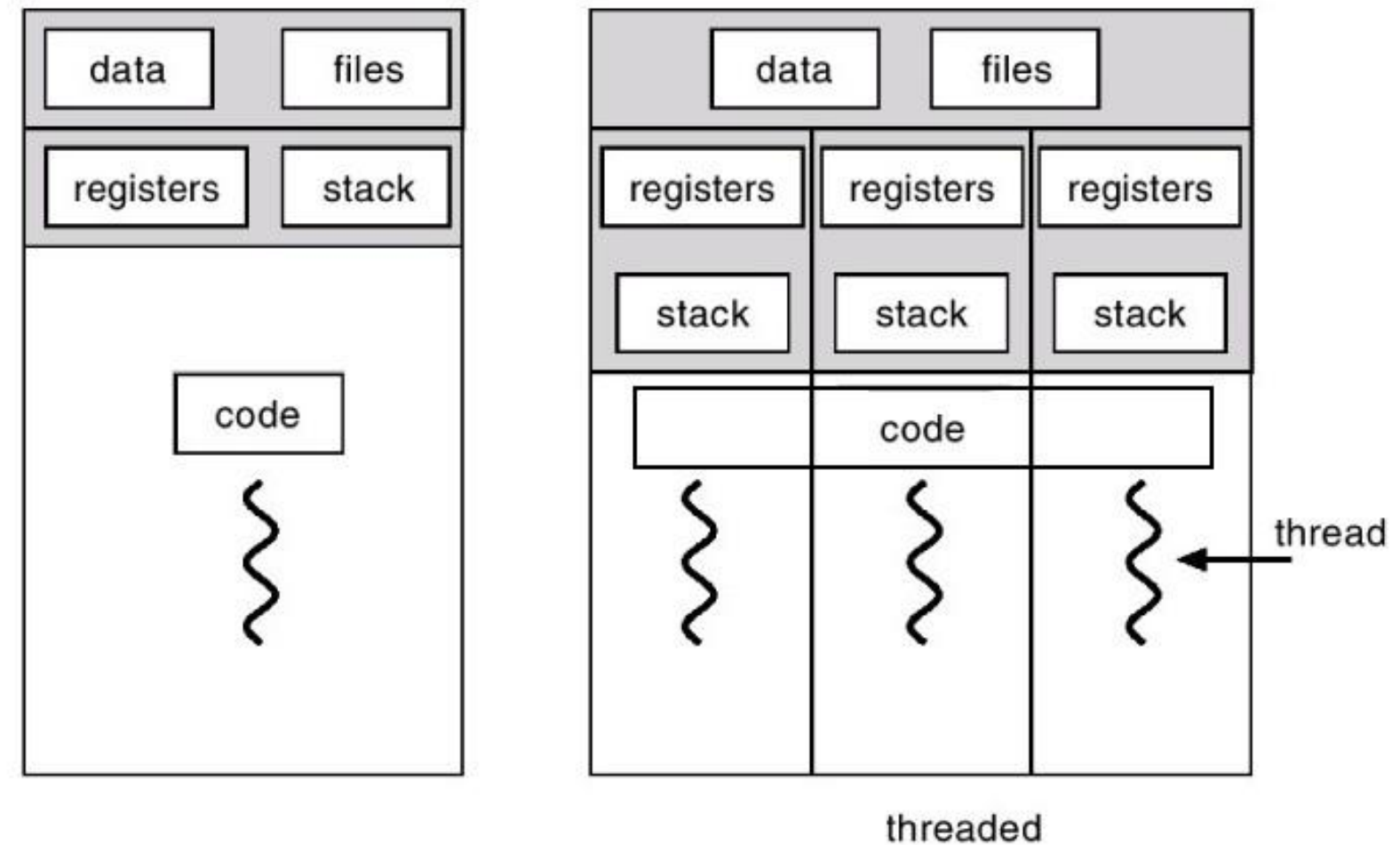
Process vs. Threads

- What is the difference between multiple process and multiple threads?



Process vs. Threads

- What is the difference between multiple process and multiple threads?
 - Each process has a complete set of its own variables
 - Threads share the same data.
 - Communication between threads is more efficient and easier to program because of shared variables.
 - Threads are more lightweight than processes.



Multithreading



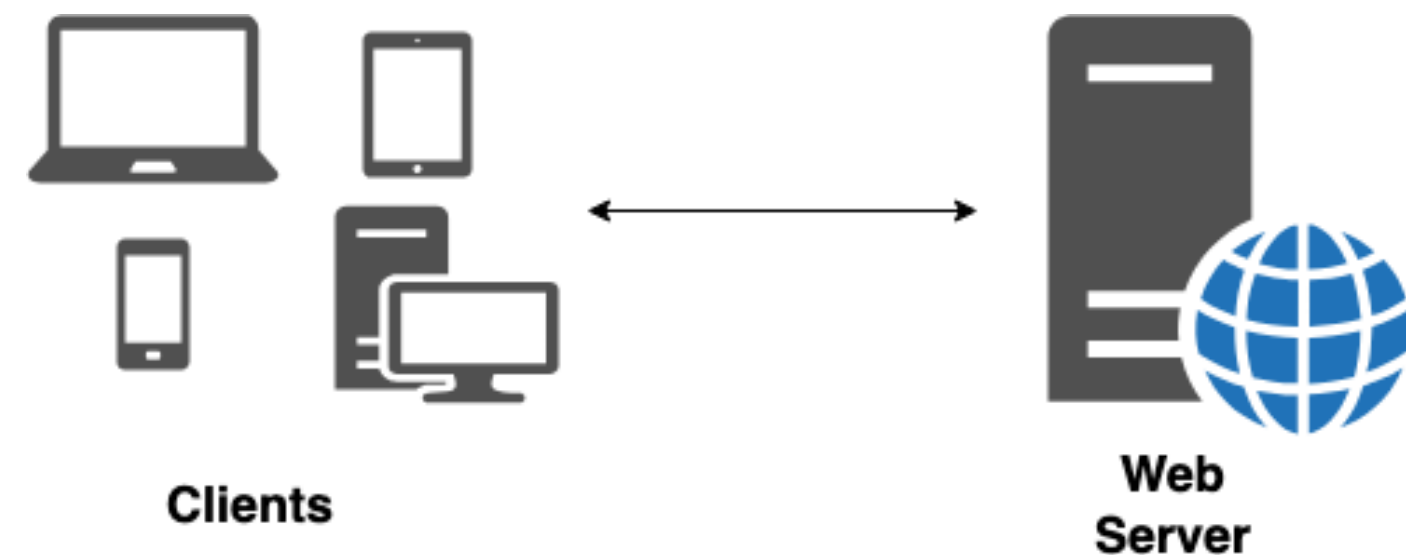
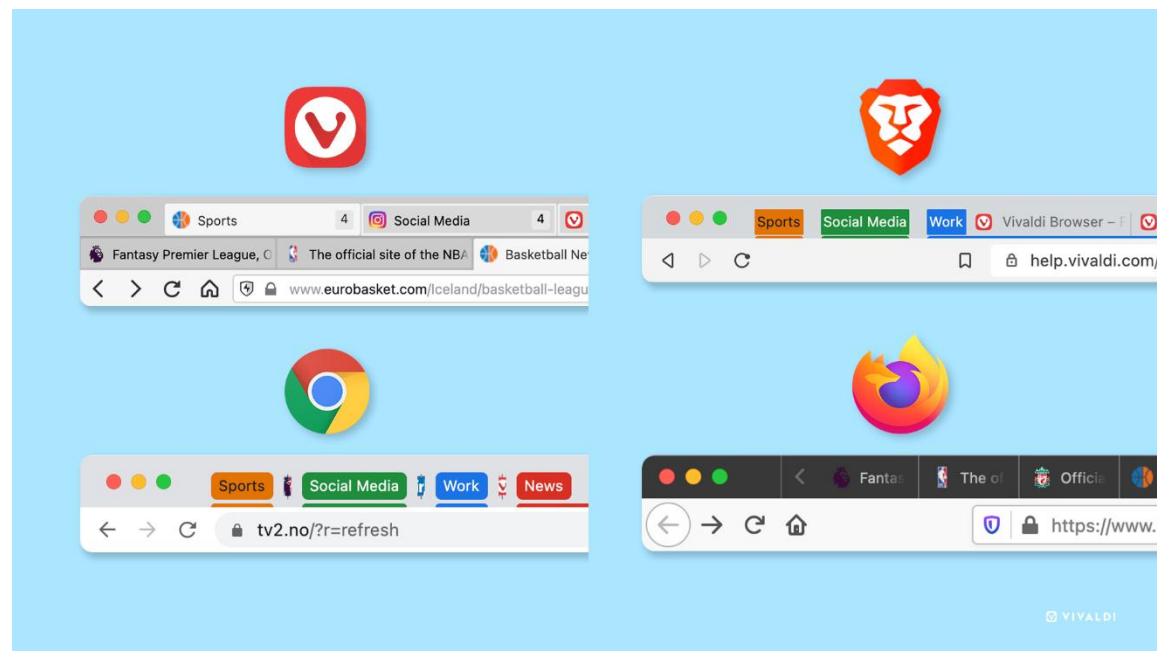
Why is Multithreading useful?

- We can perform multiple tasks at the same time.



Why is Multithreading useful?

- We can perform multiple tasks at the same time.
- **Example:**
 - A browser can download multiple webpages.
 - A web server can serve requests coming from different users.
 - Graphical User Interface programs have a separate thread for collecting user interface events from the host operating system.

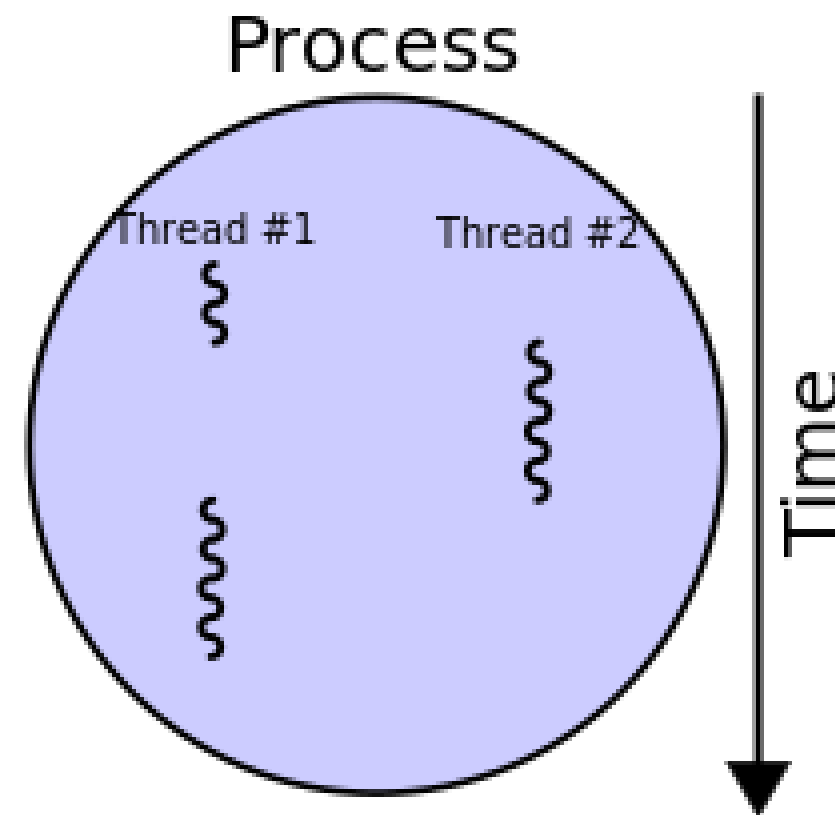


Threads



What is a thread?

- A **thread** of execution is the smallest sequence of programmed instructions that can be managed independently by a scheduler, which is typically a part of the operating system.
- In many cases, a thread is a component of a process.



The capability of running threads concurrently is called multithreading

Creating a Thread in Java

```
public interface Runnable {  
    void run();  
}
```

- The Runnable interface describes the tasks you need to run in a thread
- Multiple threads can concurrently on different cores or time slices on the same core



Banking: Concurrency needed

```
Bank.java ThreadTestBank.java
1 package com.sd.thread;
2
3 import java.util.Arrays;
4
5 //A bank with a number of bank accounts.
6 public class Bank {
7
8     private double[] accounts;
9
10    public Bank(int n, double initialBalance) {
11        accounts = new double[n];
12        Arrays.fill(accounts, initialBalance);
13    }
14
15    public void transfer(int from, int to, double amount) {
16        if(accounts[from]<amount)
17            return;
18        System.out.println(Thread.currentThread());
19        accounts[from] = accounts[from]-amount;
20        System.out.printf(" %10.2f from %d to %d", amount, from,to);
21
22        accounts[to] = accounts[to]+amount;
23        System.out.printf(" Total Balance: %10.2f\n", getTotalBalance());
24    }
25
26    //Gets the sum of all account balances.
27    public double getTotalBalance() {
28        double sum = 0;
29        for (double a : accounts)
30            sum = sum + a;
31        return sum;
32    }
33
34    //get the number of accounts in the bank
35    public int size() {
36        return accounts.length;
37    }
38
39    public static void main(String[] args) {
40        // TODO Auto-generated method stub
41    }
42
43 }
44
45
46
47
```

```
Bank.java ThreadTestBank.java
1
2
3
4
5
6
7
8 public static void main(String[] args) {
9     Bank bank = new Bank(4, 100000);
10
11     Runnable task1 = new Runnable() {
12         @Override
13         public void run() {
14             try {
15                 for(int i=0;i<STEPS;i++) {
16                     double amount = MAX_AMOUNT * Math.random();
17                     bank.transfer(0, 1, amount);
18                     Thread.sleep((int) (DELAY * Math.random()));
19                 }
20             } catch (InterruptedException e) {
21                 System.out.println("Interrupted Exception Occurs");
22             }
23         }
24     };
25
26     Runnable task2 = new Runnable() {
27         @Override
28         public void run() {
29             try {
30                 for(int i=0;i<STEPS;i++) {
31                     double amount = MAX_AMOUNT * Math.random();
32                     bank.transfer(2, 3, amount);
33                     Thread.sleep((int) (DELAY * Math.random()));
34                 }
35             } catch (InterruptedException e) {
36                 System.out.println("Interrupted Exception Occurs");
37             }
38         }
39     };
40     Thread t1 = new Thread(task1);
41     t1.setName("Task1");
42     Thread t2 = new Thread(task2);
43     t2.setName("Task2");
44     t1.start();
45     t2.start();
46 }
47
48
49
```

Part II

The Lifecycle of a Thread



Thread States



Life Cycle of a Thread

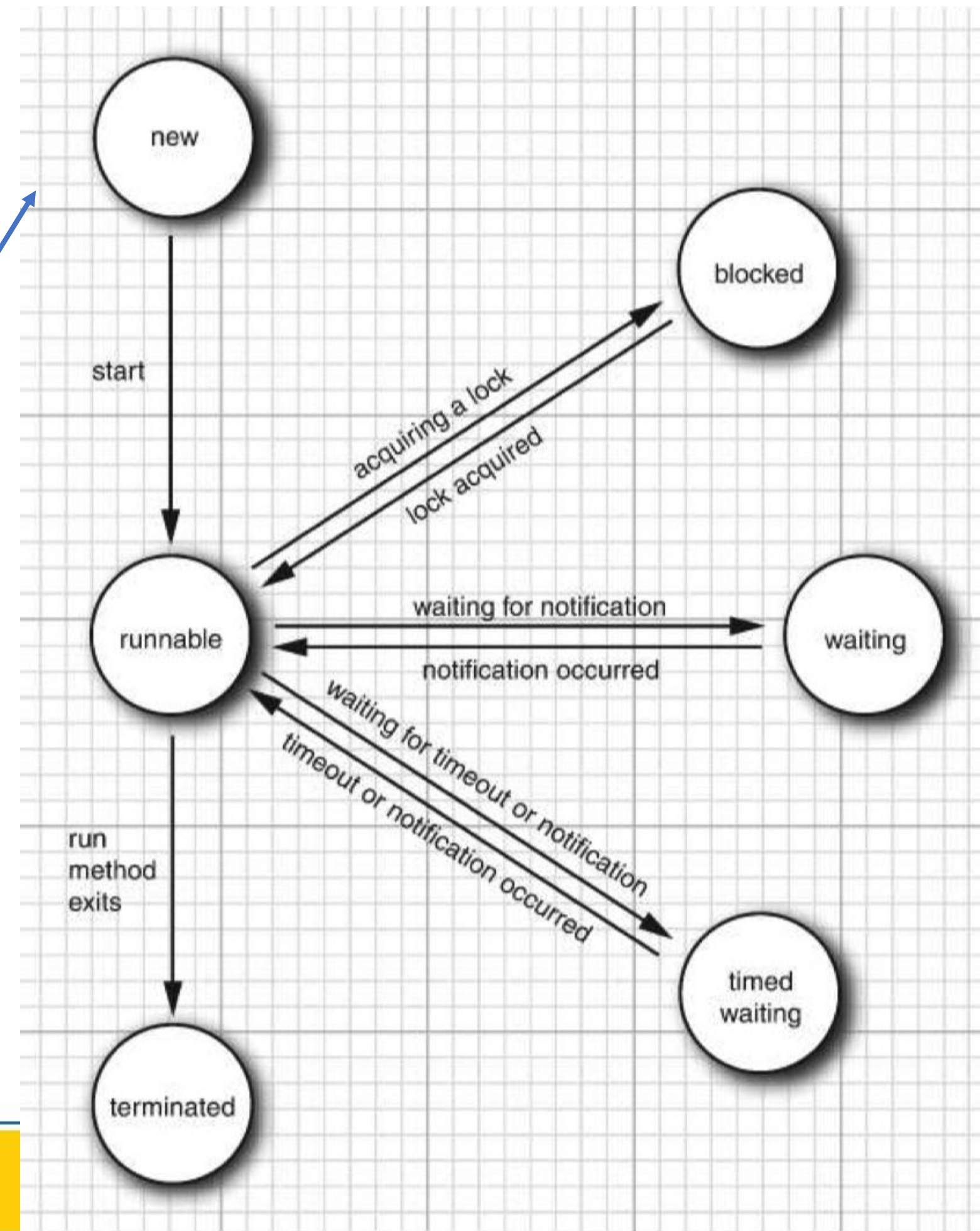
- A thread can be in one of the six states:
 - New
 - Runnable
 - Blocked
 - Waiting
 - Timed Waiting
 - Terminated



Life Cycle of a Thread

- A thread can be in one of the six states:
 - New
 - Runnable
 - Blocked
 - Waiting
 - Timed Waiting
 - Terminated

Thread t = new Thread (...)
t.setName("Task-1");



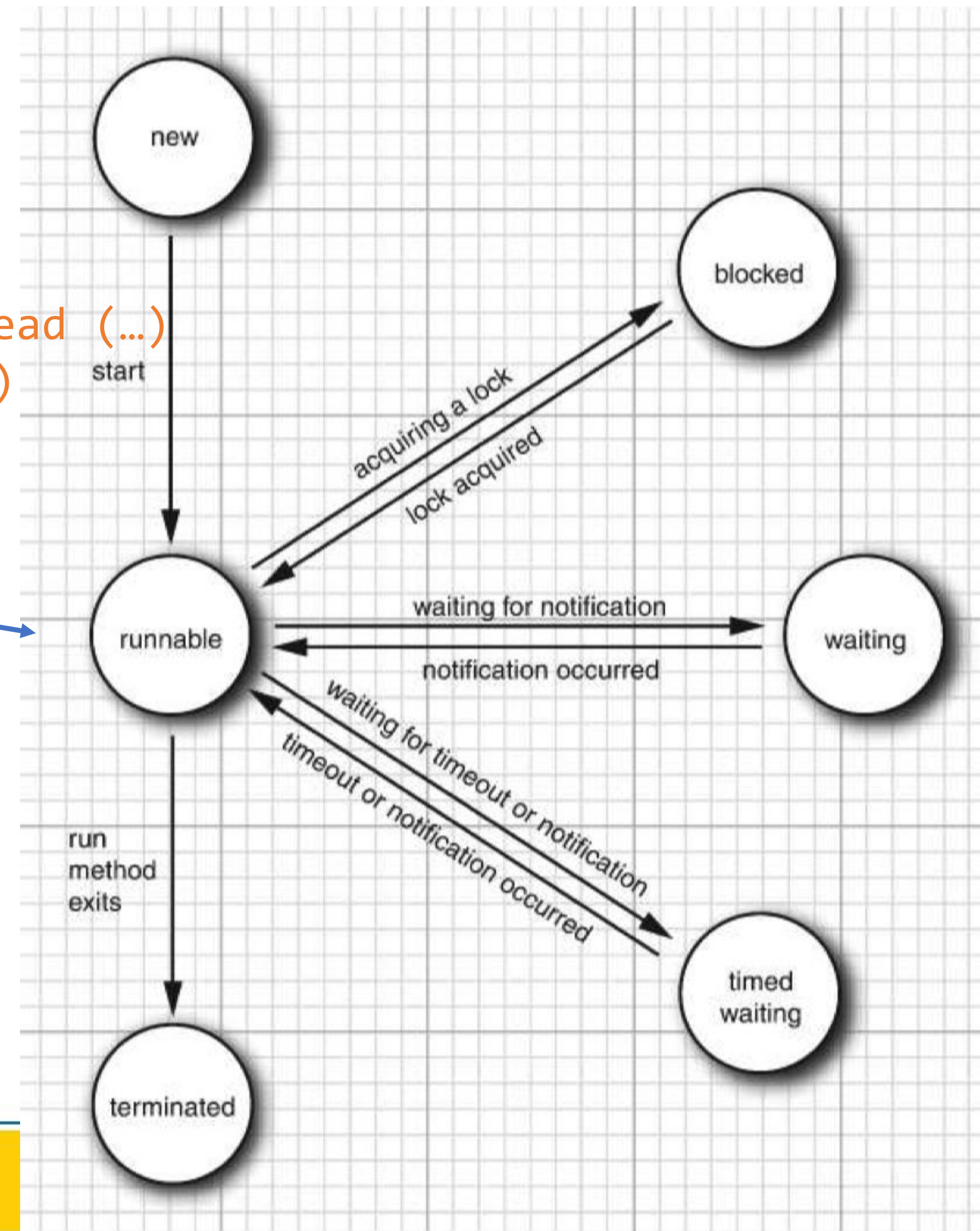
Life Cycle of a Thread

- A thread can be in one of the six states:

- New
- Runnable
- Blocked
- Waiting
- Timed Waiting
- Terminated

```
Thread T = new Thread (...)  
t.setName("Task-1")
```

```
t.start();
```



Life Cycle of a Thread

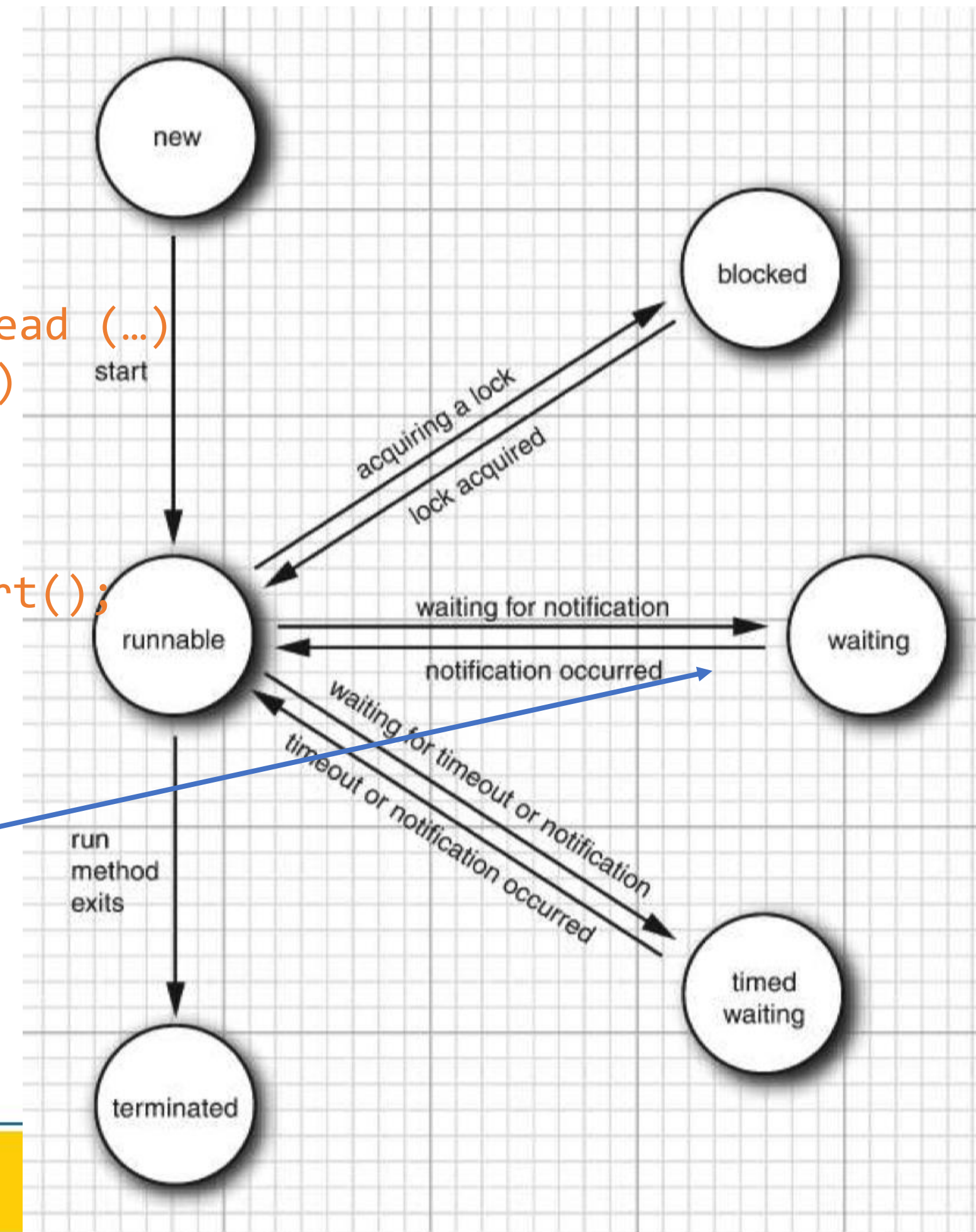
- A thread can be in one of the six states:

- New
- Runnable
- Blocked
- Waiting
- Timed Waiting
- Terminated

Thread T = new Thread (...)
t.setName("Task-1")

t.start();

Sometimes a runnable thread transitions to the waiting state while it waits for another thread to perform a task.



Life Cycle of a Thread

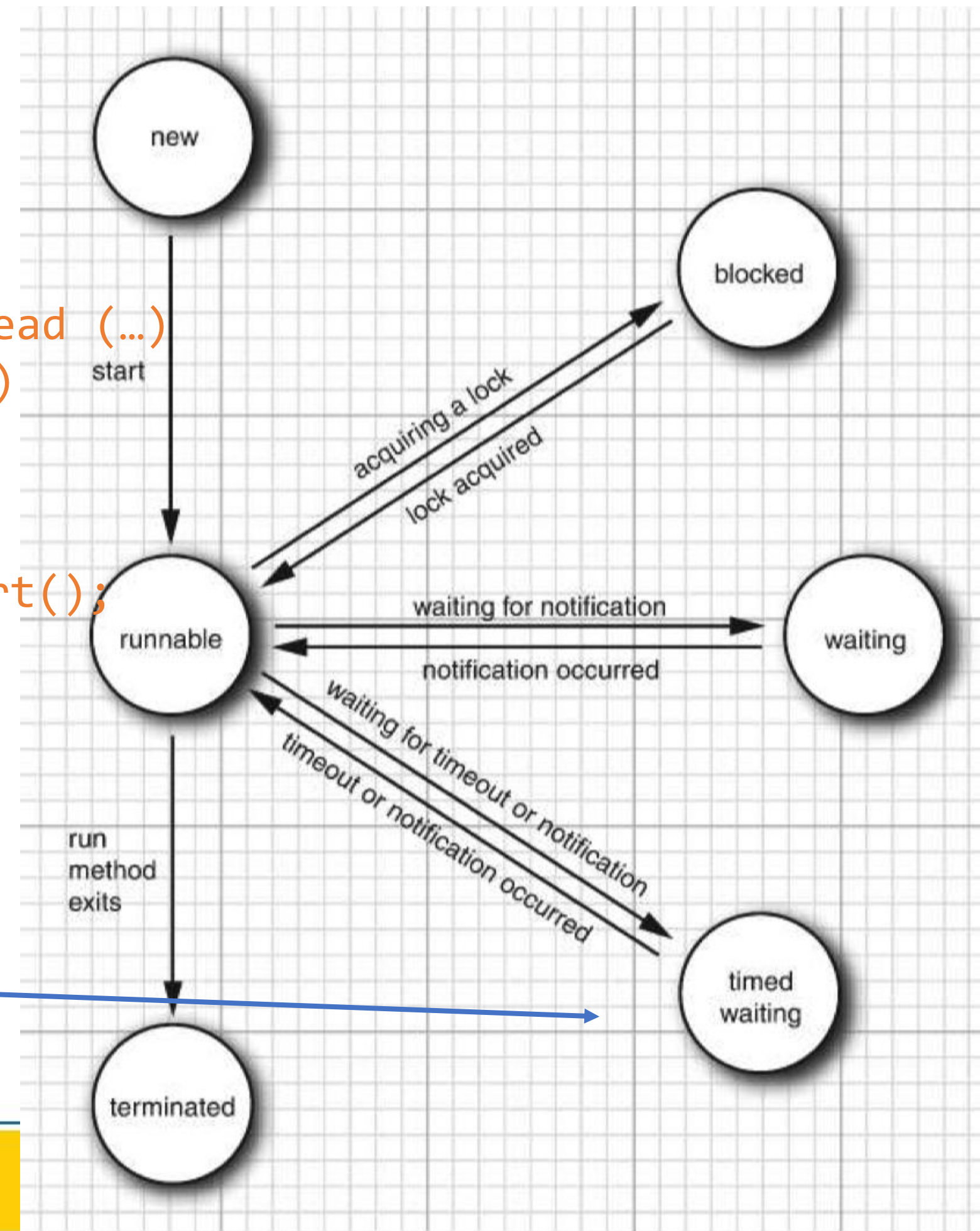
- A thread can be in one of the six states:

- New
- Runnable
- Blocked
- Waiting
- Timed Waiting
- Terminated

Thread T = new Thread (...)
t.setName("Task-1")

t.start();

Thread.sleep((int) (DELAY * Math.random()));



Life Cycle of a Thread

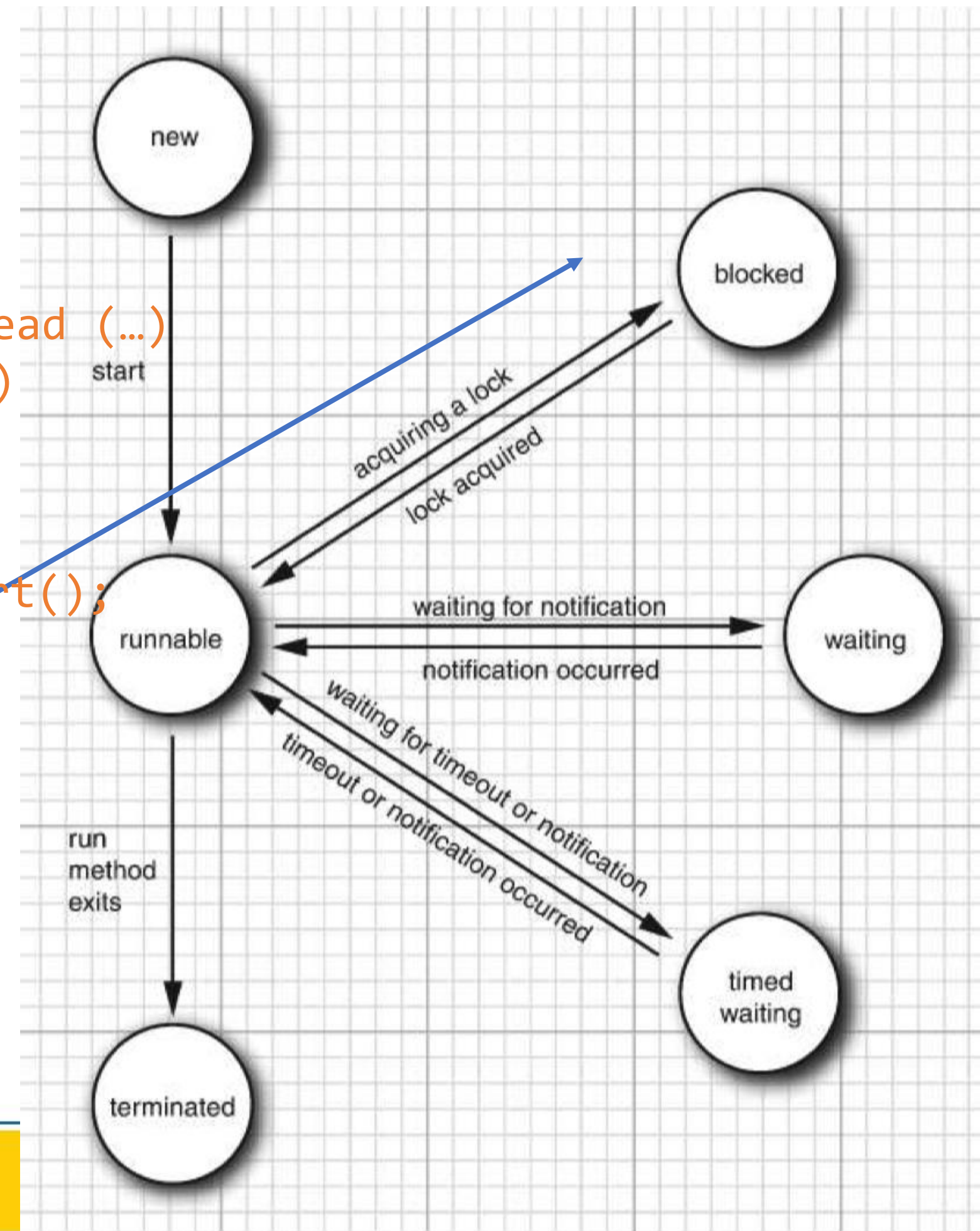
- A thread can be in one of the six states:

- New
- Runnable
- Blocked
- Waiting
- Timed Waiting
- Terminated

Thread T = new Thread (...)
t.setName("Task-1")

t.start();

A runnable thread transitions to the blocked state when it attempts to perform a task that cannot be completed immediately, and it must temporarily wait until that task completes.



Life Cycle of a Thread

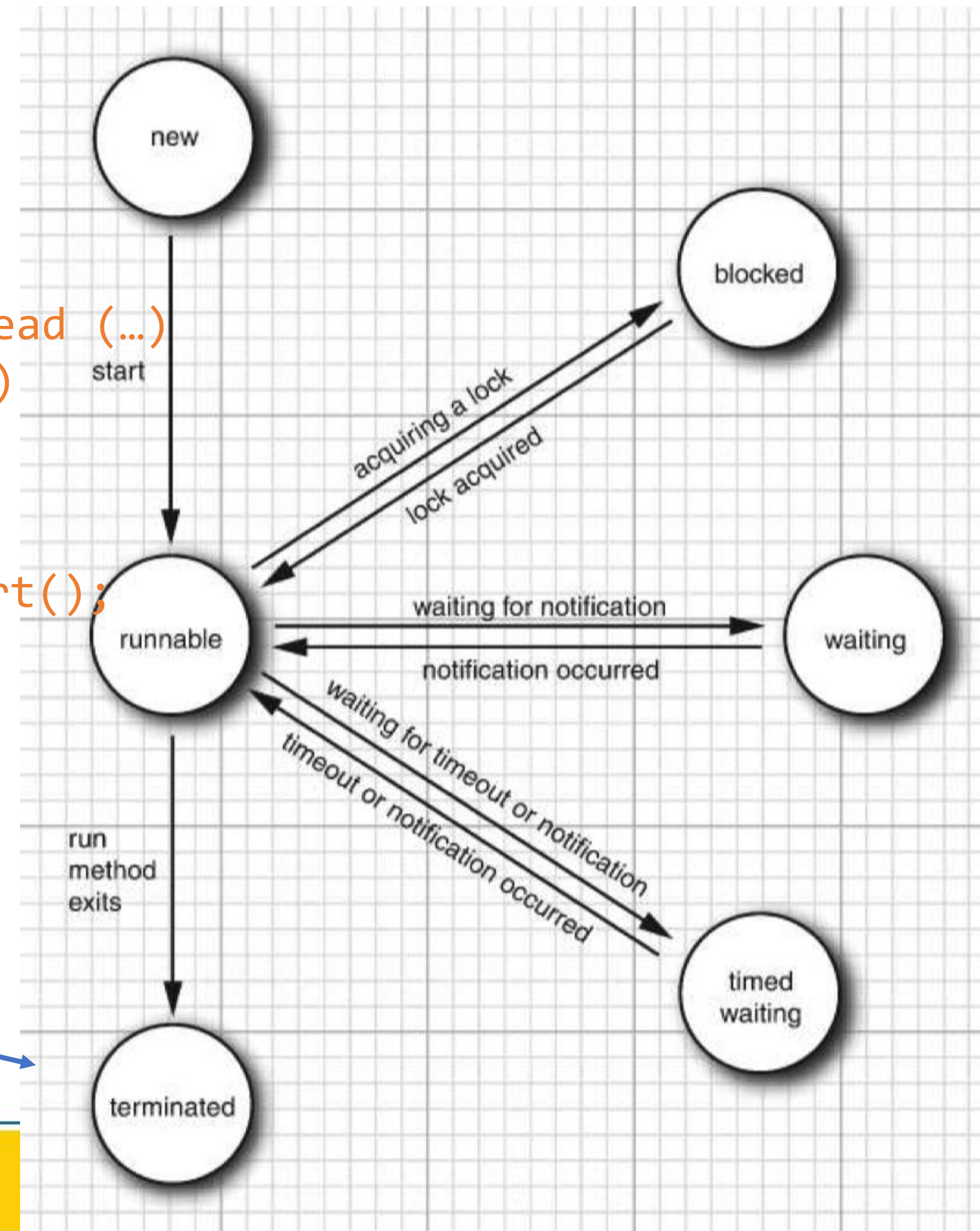
- A thread can be in one of the six states:

- New
- Runnable
- Blocked
- Waiting
- Timed Waiting
- Terminated

Thread T = new Thread (...)
t.setName("Task-1")

t.start();

A thread that completes execution or otherwise terminates enters in the terminated state



Thread Priorities



Thread Priorities

- In Java programming language every thread has a priority.
- You can increase or decrease the priority of any thread with the `setPriority()` method.
- The Thread priority level varies from 0 to 10 (MAX_PRIORITY, defined as 10. MIN_PRIORITY, defined as 1 and NORM_PRIORITY, defined as 5).
- The Thread scheduler will pick a thread with highest priority, but this is highly system dependent.



Thread Priorities

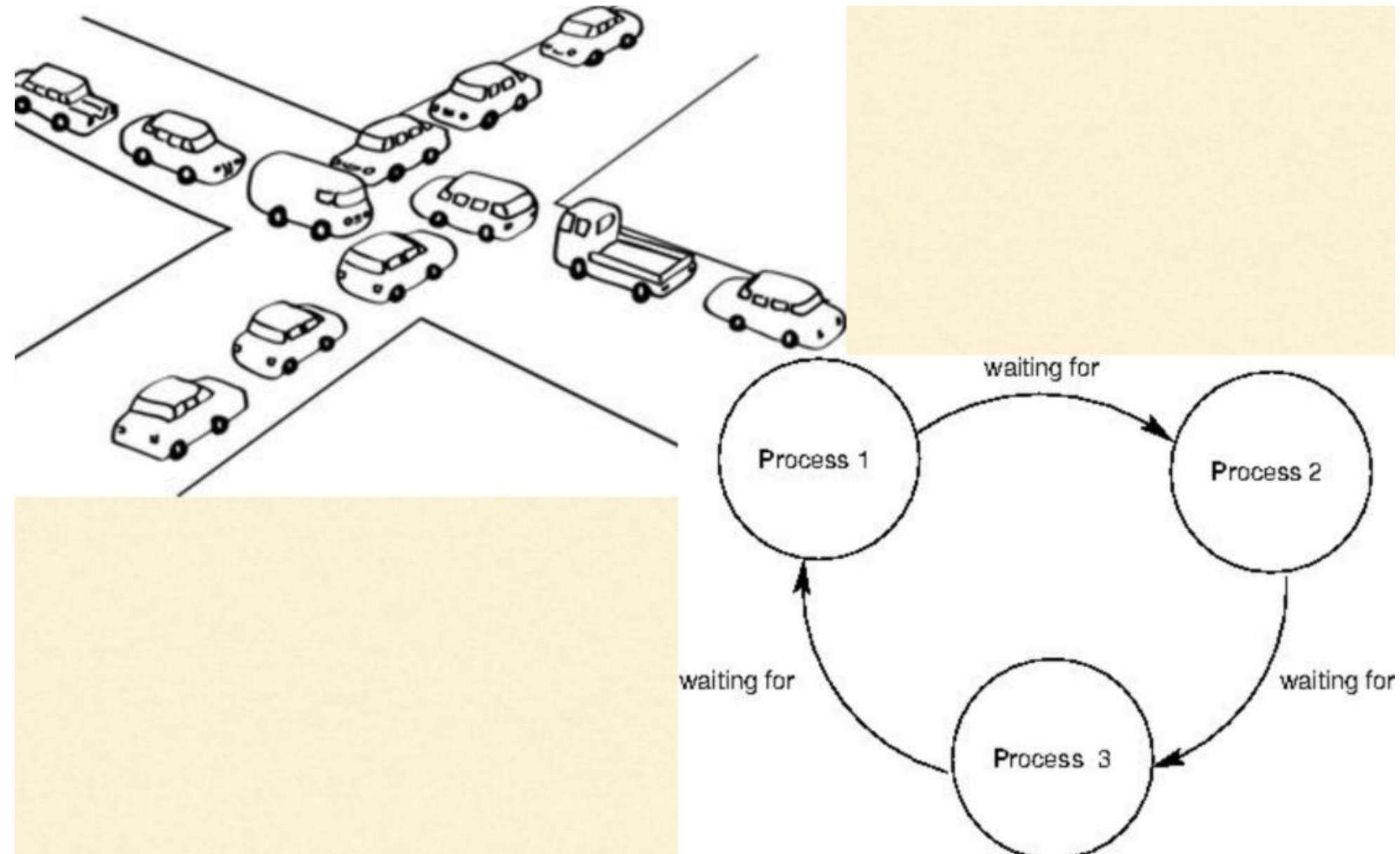
- In Java programming language every thread has a priority.
- You can increase or decrease the priority of any thread with the `setPriority()` method.
- The Thread priority level varies from 0 to 10 (MAX_PRIORITY, defined as 10. MIN_PRIORITY, defined as 1 and NORM_PRIORITY, defined as 5).
- The Thread scheduler will pick a thread with highest priority, but this is highly system dependent.
- **Thread priorities may have been useful in early versions of Java that didn't use operating systems threads.**
- **We not usually using them nowadays.**



Thread Race Condition



Race Condition



Race Condition

```
accounts[to] += amount;
```

This is not an atomic operation. Multiple steps are involved:

- Load `accounts[to]` into a register.
- Add amount.
- Move the result back to `accounts[to]`.

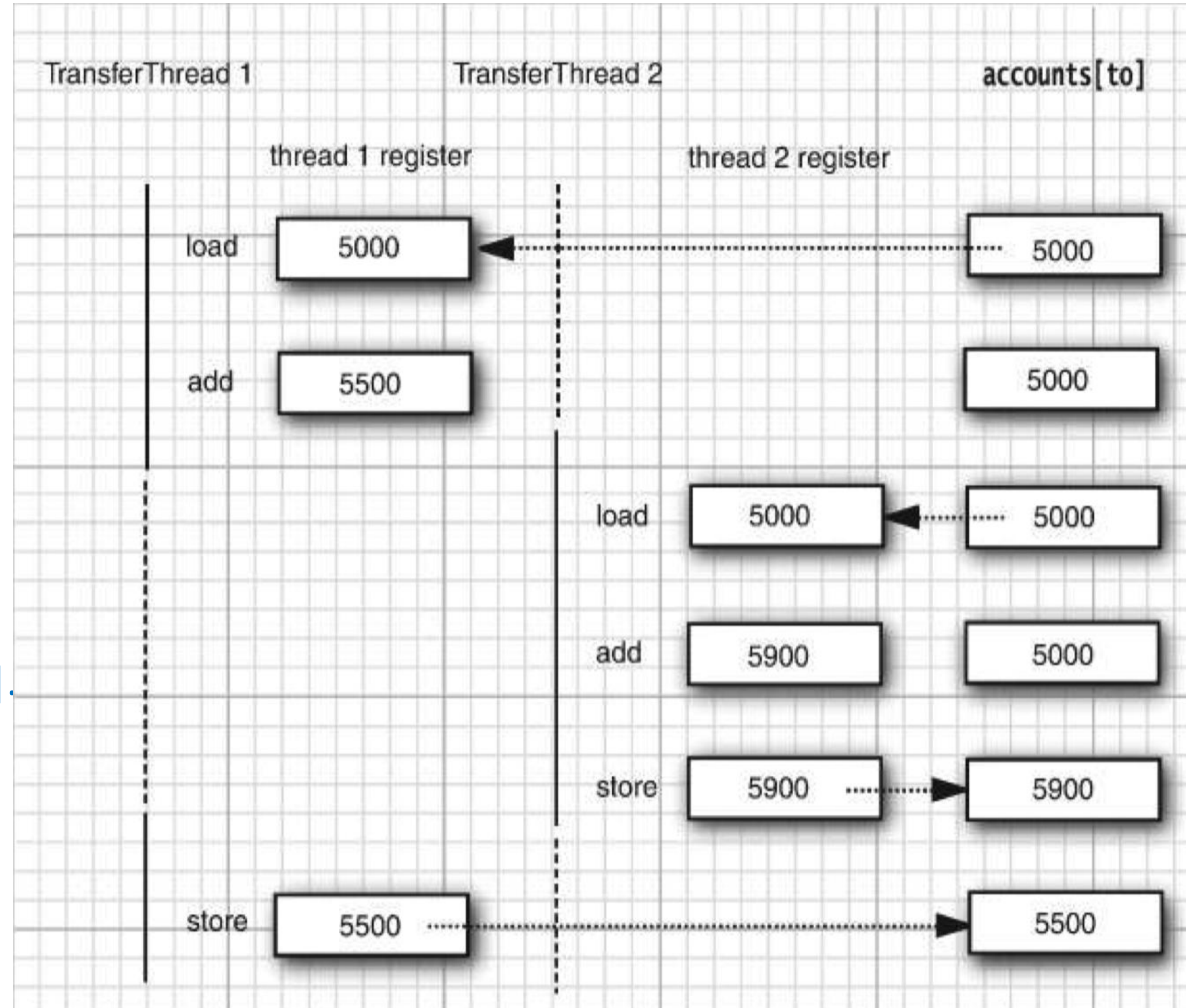


Race Condition

`accounts[to] += amount;`

This is not an atomic operation. Multiple steps are involved:

- Load `accounts[to]` into a register.
- Add amount.
- Move the result back to `accounts[to]`.



Part III

What's Next



University of Windsor

Next Lecture



University of Windsor

Next lecture

- Next lecture includes:
 - Java Concurrency (cont.)
 - Banking scheduling example
- Overview of Group Project



Thank you!
Questions?

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor